ASSIGNMENT 3 – TASK 2

PAGING: ADDRESS TRANSLATION, PAGE TABLES AND EFFECTIVE ACCESS TIME

1. Introduction and Problem Context

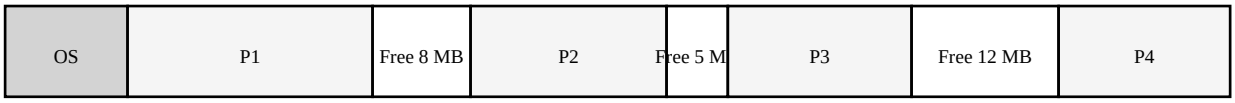
The assignment describes a multi-user college server that runs hundreds of student programs of different sizes. Contiguous memory allocation creates small unusable free gaps, and a large program may fail to load even when the total free memory is sufficient. Paging addresses this external-fragmentation problem by splitting a process's logical address space into fixed-size pages and physical memory into equal-size frames.

This report follows Q2 – Paging (8 marks, K5 – Evaluate). It covers logical-to-physical translation, page-table size, effective access time, page-size trade-offs, page-table organisations, protection and shared pages, a 100-process internal-fragmentation experiment, Linux page-size/page-fault inspection, boundary tests and the inverted-page-table alternative.

2. External Fragmentation in Contiguous Allocation

External fragmentation occurs when free memory is divided into separated holes. The sum of the holes may be large enough for a request, but no individual hole is large enough to satisfy a contiguous allocation.

Illustrative contiguous physical memory (not to scale)



Free holes: 8 + 5 + 12 = 25 MB total; largest contiguous hole = 12 MB.

A 20 MB contiguous request fails, although total free memory is 25 MB.

In the example, free memory totals 25 MB, but the largest hole is 12 MB. A process requesting 20 MB cannot be placed contiguously. Paging allows the process's pages to be placed in any available frames, so physical contiguity is not required.

3. Logical-to-Physical Address Translation

For a page size S, a logical address A is divided into a page number p and an offset d:

p = floor(A / S)
d = A mod S

The page table maps page p to a physical frame f. If p is outside the page table, the address is invalid. If the entry is −1, the page is not resident and a page fault occurs. Otherwise:

Physical address = f × S + d

3.1 Assignment data and completed TODO

The supplied program uses PAGE_SIZE = 1024, PAGES = 8 and page_table = {5, 2, −1, 7, 0, −1, 3, −1}. The completed code splits each address into page/offset, checks the page-table bounds, detects a non-resident page and calculates the physical address when a frame is present.

Logical address	Page	Offset	Page-table entry	Result
0	0	0	5	5 × 1024 + 0 = 5120
1500	1	476	2	2 × 1024 + 476 = 2524
3100	3	28	7	7 × 1024 + 28 = 7196
6200	6	56	3	3 × 1024 + 56 = 3128
9000	8	808	Outside 0–7	Invalid address
2049	2	1	–1	Page fault

The program's output matches the expected output in the question paper. The C file also tests the first and last byte of a page, the first byte of the next page, the last address represented by the page table and the address immediately beyond the table.

3.2 Recalculation for PAGE_SIZE = 2048

When PAGE_SIZE changes to 2048, the logical page and offset change. The same eight-entry page table is used, so page numbers 0–7 remain addressable.

Logical address	Page	Offset	Frame	Physical result
0	0	0	5	10240
1500	0	1500	5	11740
3100	1	1052	2	5148
6200	3	56	7	14392
9000	4	808	0	808
2049	1	1	2	4097

Unlike the 1 KB case, all six addresses are valid and resident with 2 KB pages. This illustrates that changing page size changes page-number/offset decomposition and therefore changes the page-table entry consulted.

4. Page-Table Size for 32-bit and 48-bit Address Spaces

A 4 KB page contains 2^12 bytes. The number of virtual pages in a single-level page table is 2^(address bits − 12). The assignment does not specify the size of a page-table entry, so the main calculation assumes a 4-byte page-table entry for both address widths. The table also shows the consequence of an 8-byte entry.

Virtual address width	Virtual page number bits	Entries in one-level table	Size at 4-byte PTE	Size at 8-byte PTE
32-bit	32 – 12 = 20	2^20 = 1,048,576	4 MiB	8 MiB
48-bit	48 – 12 = 36	2^36 = 68,719,476,736	256 GiB	512 GiB

Calculations: 2^20 × 4 bytes = 4,194,304 bytes = 4 MiB. 2^36 × 4 bytes = 274,877,906,944 bytes = 256 GiB. These are theoretical single-level page-table sizes if every virtual page has an entry. A 48-bit process-wide flat table is impractically large; hierarchical page tables allocate lower-level tables only for populated address-space regions. Actual PTE width depends on architecture and implementation, which is why the entry-size assumption is stated.

5. Effective Memory-Access Time and the TLB

A Translation Lookaside Buffer (TLB) caches recently used page-table entries. A TLB hit needs a TLB lookup and one memory access for the requested data. A TLB miss needs the TLB lookup, a page-table memory access (assuming a single-level table) and the data memory access.

Assumptions for the numerical example (not supplied by the question): TLB lookup time t = 10 ns, main-memory access time m = 100 ns, and no page fault.

Without a TLB: EAT = 2m = 200 ns.
With a TLB: EAT = h(t + m) + (1 – h)(t + 2m), where h is the TLB hit ratio.

TLB hit ratio	Hit time	Miss time	Effective access time
80%	110 ns	210 ns	0.80(110) + 0.20(210) = 130 ns
95%	110 ns	210 ns	0.95(110) + 0.05(210) = 115 ns
99%	110 ns	210 ns	0.99(110) + 0.01(210) = 111 ns

Higher hit ratios reduce the average translation overhead. A real multi-level page-table walk can require more than one memory access on a TLB miss; this simplified example assumes a single-level page table and excludes page-fault service time.

6. Evaluation of Page Sizes: 1 KB, 4 KB and 2 MB

Page size	Advantages	Disadvantages / suitability
1 KB	Small final-page waste for many small programs; fine-grained allocation.	More pages per process, larger page tables and greater TLB pressure; useful where minimising internal waste is more important.
4 KB	Balanced granularity; common general-purpose system page size; moderate page-table size and internal waste.	More page-table entries than huge pages; some final-page internal fragmentation remains.
2 MB	Fewer pages and page-table entries; can improve TLB reach and help large sequential memory workloads.	Potentially large internal fragmentation for small processes; allocation/compaction and NUMA constraints may complicate use.

For a multi-user college server running hundreds of programs of different sizes, 4 KB is a reasonable default compromise: much less internal waste than 2 MB pages while avoiding the very large page-table/TLB overhead associated with very small pages. Huge pages of 2 MB can be used selectively for large, long-lived workloads when measurements show a TLB benefit and memory waste is acceptable. The server's actual base page size must be checked using getconf PAGESIZE rather than assumed.

7. Hierarchical, Hashed and Inverted Page Tables

Organisation	Structure and translation	Evaluation
Hierarchical / multi-level	Virtual page number is split into fields. The hardware/OS walks a top-level entry and subsequent lower-level tables until the frame entry is found.	Allocates lower levels only where needed, reducing space for sparse address spaces. A TLB miss requires a walk of multiple levels.
Hashed page table	A hash of (process ID, virtual page number) locates a chain/bucket containing the frame mapping.	Useful for large sparse address spaces; average lookup can be efficient, but collisions/chain traversal add cost.

Organisation	Structure and translation	Evaluation
Inverted page table	One entry per physical frame, storing the process ID and virtual page mapped to that frame. A hash index can accelerate searches.	Space is proportional to physical frames rather than virtual pages per process, but translation requires lookup by process and virtual page; shared pages and collisions need additional handling.

A multi-level page table uses O(number of allocated page-table entries) space, with a worst-case single-level upper bound determined by virtual address width. A page walk takes O(L) levels on a TLB miss, where L is the number of page-table levels. An inverted table stores O(F) entries for F physical frames; a hash lookup is expected O(1) and can degrade to O(F) in the worst case. The included inverted_page_table_demo.c illustrates a hash lookup keyed by (PID, virtual page).

8. Protection and Shared Pages

Each page-table entry can contain protection and status information in addition to the frame number. Common permissions include read, write and execute, together with user/supervisor access, present/valid, accessed and dirty bits. The MMU checks permissions on each memory reference. An illegal access can raise a protection fault, which the operating system handles (often by terminating the process or delivering a signal).

Shared pages are provided by mapping the same physical frame into the page tables of multiple processes. Read-only program code and shared libraries can be mapped into several address spaces while each process has its own private writable data and stack. Copy-on-write allows processes to share a page until one attempts to modify it; the OS then creates a private copy and updates the relevant page-table entry.

9. Experimental Investigation: Internal Fragmentation

The supplied paging_experiments.c program generates 100 deterministic pseudo-random process sizes from 1 byte through 10 MiB using a fixed seed. For each page size, a process is allocated ceil(process size / page size) pages. Internal waste is allocated bytes minus requested bytes, accumulated across all 100 processes.

Waste per process = ceil(process size / page size) × page size − process size

Page size	Total requested bytes	Total allocated bytes	Total internal waste	Average waste/process
1 KB (1,024 B)	493,876,974	493,931,520	54,546	545.46 B
4 KB (4,096 B)	493,876,974	494,092,288	215,314	2,153.14 B
2 MB (2,097,152 B)	493,876,974	599,785,472	105,908,498	1,059,084.98 B

These results are reproducible for the supplied fixed seed and range. They illustrate the trade-off: larger pages reduce the number of pages but increase potential internal fragmentation in the final page of each process. These values are for the generated test set, not a measurement of real college-server workloads.

10. Linux Page Size and Page-Fault Measurement

The run script records the native Linux page size with getconf PAGESIZE. It also compiles paging_memory_probe.c, which allocates 64 MiB and writes one byte in each native page. /usr/bin/time -v captures minor and major page-fault counts, maximum resident set size and elapsed wall-clock time.

Measurement	Where to find it
Native page size	linux_page_size.txt and environment.txt
Minor / major page faults	page_faults_time.txt and page_fault_summary.txt
Maximum resident set size	page_faults_time.txt
Touched-memory program output	paging_memory_probe_output.txt

Run the script on the Ubuntu/WSL system used for submission and record its actual page size and page-fault counts. Page-fault counts vary with kernel, memory state, allocator, execution environment and system load; do not copy values from another machine.

11. Complete-the-Code Program and Boundary Tests

The submitted paging_translation.c completes TODO 1 while retaining the assignment's data. It also includes tests at the start and end of a page and immediately beyond the process's eight-page table.

```
For PAGE_SIZE = 1024:
0     -> page 0 offset 0       : physical 5120
1023  -> page 0 offset 1023    : physical 6143
1024  -> page 1 offset 0       : physical 2048
8191  -> page 7 offset 1023    : page fault
8192  -> page 8 offset 0       : invalid address
```

The address 8191 is inside the logical range covered by the eight-page table but page 7 is not resident (entry −1), so it produces a page fault. Address 8192 is beyond the table's logical range and is invalid. These are different conditions: a page fault means a valid logical page is not currently resident; an invalid address is outside the process's defined page-table range.

12. Alternative Concept: Inverted Page Table

With a per-process multi-level page table, each process maintains translation entries for the virtual pages it uses. An inverted page table instead has one entry for each physical frame in the system; each entry identifies which (PID, virtual page number) currently occupies that frame. A lookup hashes the PID and virtual page number to find the corresponding physical frame. The included inverted_page_table_demo.c is a simplified demonstration with a small open-addressed hash table.

Aspect	Per-process multi-level page table	Inverted page table
Space	Depends on virtual pages mapped by each process; lower-level tables can be allocated sparsely.	O(F) entries total, where F is the number of physical frames.
Lookup	Walk L page-table levels on a TLB miss: O(L) table levels.	Hash lookup expected O(1); worst-case search can be O(F).
Scaling	Large virtual address spaces can require many lower-level tables when densely mapped.	Space scales with physical memory, not the total virtual address space of every process.
Complexity	Structured hardware page walk; TLB caches translations.	Hashing and PID tagging are needed; shared mappings and collisions require handling.

For a large multi-user server with sparse 48-bit virtual address spaces, an inverted page table can reduce metadata space because it stores one entry per physical frame. The cost is more complicated lookup and management, especially for shared pages. A multi-level table is usually simpler and benefits from hardware TLB/page-walk support. The final design depends on memory pressure, architecture support and measured translation overhead.

13. Conclusion and Recommendation

Paging eliminates external fragmentation because pages can occupy any free physical frames, but it retains internal fragmentation in the last page of each process. The supplied address examples demonstrate frame lookup, page faults and invalid addresses. The calculations show that single-level page tables can become extremely large for wide virtual address spaces, motivating hierarchical, hashed or inverted designs. A TLB substantially reduces effective access time as its hit ratio increases. For a multi-user college server, 4 KB base pages are a balanced default, while 2 MB huge pages may be enabled selectively for large workloads. The final choice should be validated using the server's workload distribution, native page size, page-fault behaviour and measured TLB performance.

14. Execution Procedure and Evidence

1. Place paging_translation.c, paging_experiments.c, inverted_page_table_demo.c, paging_memory_probe.c and run_task2.sh in one Task02 folder.
2. Open Ubuntu/WSL in that folder.
3. Run chmod +x run_task2.sh.
4. Run ./run_task2.sh.
5. Verify the six expected 1 KB translation results and 2 KB recalculation.
6. Review the 100-process internal-fragmentation and TLB results.
7. Review getconf PAGESIZE and /usr/bin/time -v page-fault output.
8. Save generated text outputs and capture terminal screenshots.

Evidence checklist: external-fragmentation diagram; page translation program and output; 32-/48-bit page-table calculations; TLB EAT calculations; page-size evaluation; 100-process fragmentation results; Linux native page size and page-fault evidence; page-boundary tests; inverted-page-table alternative and complexity comparison.

Source alignment: Q2 – Paging, 8 Marks, K5 – Evaluate, pages 23–24 of the supplied Assignment 3 question paper.